Universidad Rafael Landivar

Facultad de Ingeniería

Licenciatura en Ingeniería en Informática y Sistemas

Manejo e Implementación de Archivos, sección 02

Docente: Ing. Sergio Raúl Torres Barrios

"LABORATORIO 2"

Dennys de Jesús Aguilar Mérida – 1588425

Marlon Esteban Roblero Ordoñez – 1524625

Diego André Escobar Rodríguez - 1579925

1. Introducción

El programa desarrollado en Python permite administrar una agenda de contactos mediante una interfaz gráfica. Los contactos se almacenan en un archivo binario de tamaño fijo. Además, el programa utiliza un índice para facilitar las búsquedas, calcula un hash SHA-256 para verificar la integridad de los datos y permite crear un respaldo ZIP.

El objetivo es explicar cómo se organiza cada registro, cómo se accede directamente a su posición en el archivo y cómo se utilizan el índice, la verificación de integridad y el respaldo.

2. Estructura del registro

El archivo contactos.dat guarda los datos de cada contacto en registros de tamaño fijo. En el código, las longitudes se definen en la lista CAMPOS. Al final se agrega un salto de línea de un byte.

Campo	**Longitud**
ID	8 bytes
Nombre	30 bytes
Teléfono	15 bytes
Correo electrónico	26 bytes
Ciudad	20 bytes
Salto de línea	1 byte
TOTAL	**100 bytes**

El tamaño total se calcula de la siguiente manera:

8 + 30 + 15 + 26 + 20 + 1 = 100 bytes

La función ajustar_campo() adapta cada valor a la longitud asignada. La función codificar_elregistro() une los campos y agrega el salto de línea. Finalmente, una comprobación assert verifica que el registro tenga exactamente 100 bytes.

```
30  CAMPOS = [("id", 8), ("nombre", 30), ("telefono", 15), ("correo", 26), ("ciudad", 20)]
31
32  ETIQUETAS = {"id": "ID", "nombre": "Nombre", "telefono": "Teléfono", "correo": "Correo", "ciudad": "Ciudad"}
33
34  LEN_DATOS = sum(largo for _, largo in CAMPOS)
35
36  TAMANIO_DELREGISTRO = LEN_DATOS + 1
37  assert TAMANIO_DELREGISTRO == 100
```

3. Acceso aleatorio con seek() y tell()

El acceso aleatorio permite ir directamente a una posición del archivo sin leer primero todos los registros anteriores. En Python, seek() mueve el cursor del archivo y tell() devuelve la posición actual en bytes.

3.1. Agregar un contacto

```
with open(ARCHIVO_DATOS, modo) as f:
    f.seek(0, os.SEEK_END)
    pos = f.tell()
    num = pos // TAMANIO_DELREGISTRO
    f.write(codificar_elregistro(valores))
```

seek(0, os.SEEK_END) coloca el cursor al final del archivo. tell() obtiene la posición en bytes y la división entera entre 100 calcula el número del siguiente registro. Después, write() escribe el nuevo contacto.

3.2. Buscar un contacto

```
        num = self.indice[id_txt]
        try:
            with open(ARCHIVO_DATOS, "rb") as f:
                f.seek(num * TAMANIO_DELREGISTRO)
                raw = f.read(TAMANIO_DELREGISTRO)
                valores = decodificar_elregistro(raw)
```

El programa consulta el índice para obtener el número del registro. Luego multiplica ese número por 100 para calcular su posición inicial. Por ejemplo, el registro 0 comienza en el byte 0, el registro 1 en el byte 100 y el registro 2 en el byte 200. El programa lee únicamente los 100 bytes del registro solicitado.

3.3. Modificar un contacto

```
    num = self.indice[id_txt]
    try:
        with open(ARCHIVO_DATOS, "r+b") as f:
            f.seek(num * TAMANIO_DELREGISTRO)
            f.write(codificar_elregistro(valores))
```

El modo r+b permite leer y escribir en el archivo binario existente. seek() ubica el registro y write() reemplaza su contenido sin tener que reescribir todos los demás registros.

4. Funcionamiento del índice

El programa utiliza indice.json para relacionar el ID de cada contacto con el número de registro en contactos.dat. En los archivos de prueba entregados, el índice contiene la siguiente relación:

```
{
 "03": 0
}
```

Esto indica que el contacto con ID "03" está en el registro número 0. Al buscar un contacto, el programa consulta el diccionario self.indice y usa el número encontrado para calcular la posición con seek(). Cuando se agrega un contacto, el índice se actualiza y se guarda en el archivo JSON.

Búsqueda secuencial	**Búsqueda con índice**
Revisa los registros uno por uno hasta encontrar el contacto.	Consulta el ID en el índice y accede directamente al registro.
Puede necesitar leer muchos registros.	Evita recorrer todos los registros del archivo.

Por esta razón, el índice suele hacer más eficiente la búsqueda, especialmente cuando hay muchos contactos. La velocidad total también depende de la cantidad de datos y del acceso al archivo.

5. Verificación de integridad con SHA-256

El programa calcula un hash SHA-256 del contenido de contactos.dat y guarda el resultado en contactos.sha256. Cuando se verifica la integridad, vuelve a calcular el hash y lo compara con el valor guardado.

```
@staticmethod
def calcular_hash():
    hasher = hashlib.sha256()
    with open(ARCHIVO_DATOS, "rb") as f:
        for bloque in iter(lambda: f.read(65536), b""):
            hasher.update(bloque)
    return hasher.hexdigest()
```

El archivo se lee en bloques de 65,536 bytes. hashlib.sha256() calcula el hash y hexdigest() devuelve el resultado en caracteres hexadecimales.

Si los hashes son diferentes, el programa muestra una alerta de posible modificación. Si coinciden, el archivo se considera íntegro respecto al hash guardado. El hash permite detectar cambios, pero por sí solo no impide que alguien modifique los datos. Si se acepta un archivo alterado, el programa permite recalcular el hash y reconstruir el índice.

La evidencia visual de la alteración manual se agregará por separado mediante capturas de pantalla; por solicitud, este documento no incluye capturas.

6. Comparación del archivo original y el respaldo ZIP

La opción Crear respaldo ZIP comprime contactos.dat y, si existe, indice.json. El programa obtiene el tamaño del archivo original y el del ZIP mediante os.path.getsize().

En los archivos de prueba incluidos en el ZIP proporcionado se observaron estos tamaños:

Archivo	**Tamaño**
contactos.dat (original)	100 bytes
respaldo.zip (respaldo)	295 bytes
Diferencia	**195 bytes**

En este caso, el ZIP es más grande que contactos.dat porque el archivo original es pequeño y el respaldo incluye información adicional de compresión y el índice. La comparación corresponde a los archivos entregados; al crear otro respaldo, el tamaño puede variar.

7. Funcionamiento de la aplicación

La interfaz gráfica está desarrollada con tkinter. Desde la ventana y sus menús se pueden realizar las siguientes operaciones:

Función	Descripción
Agregar contacto	Guarda un nuevo contacto en el archivo binario.
Buscar por ID	Usa el índice para localizar el registro.
Modificar contacto	Actualiza los datos de un registro existente.
Limpiar	Vacía los campos del formulario.
Exportar a JSON	Genera archivo.json con los contactos.
Generar reporte PDF	Genera un reporte de contactos si la biblioteca necesaria está disponible.
Verificar integridad	Compara el hash actual con el guardado.
Crear respaldo ZIP	Crea una copia comprimida de los datos y del índice.
Información del archivo	Muestra el tamaño, cantidad de registros y permisos de lectura y escritura.

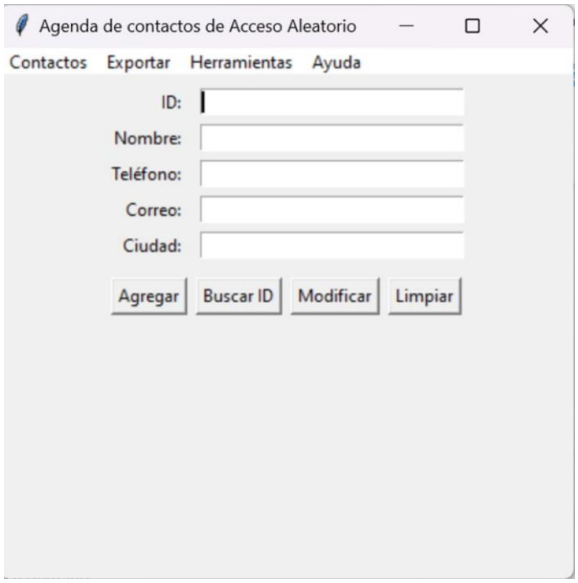
Al iniciar, la aplicación programa una verificación de integridad. También carga el índice existente o intenta reconstruirlo si es necesario. Las capturas del funcionamiento de la aplicación se agregarán por separado.

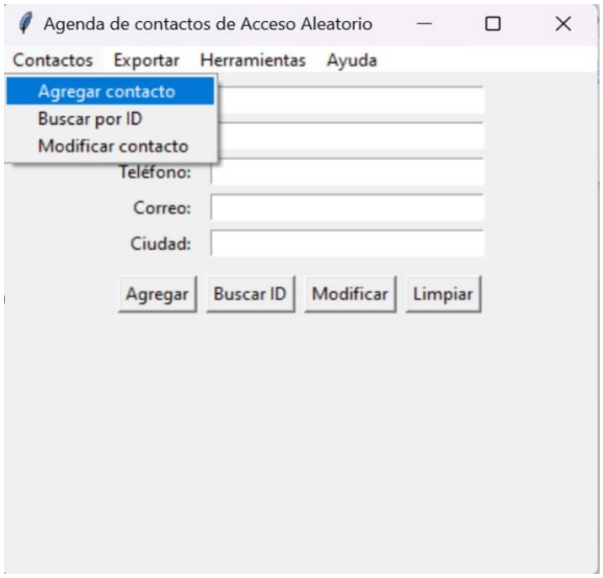
8. Conclusión

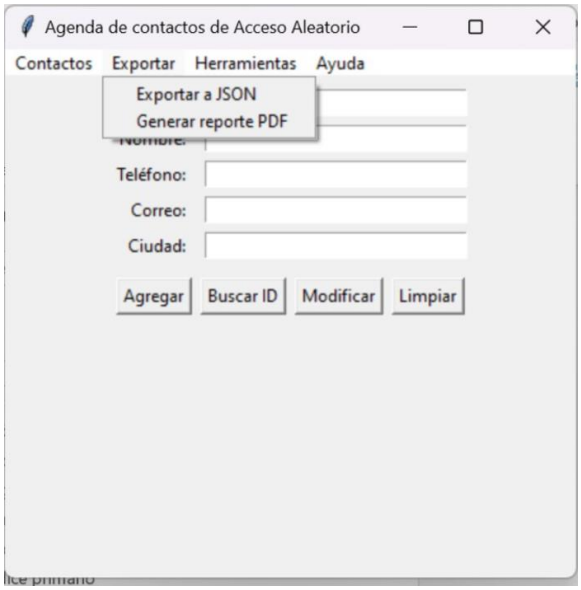
El laboratorio muestra cómo almacenar y administrar contactos mediante un archivo binario de tamaño fijo. Con seek() y tell() se puede acceder a posiciones específicas; el índice facilita las búsquedas por ID; SHA-256 ayuda a detectar cambios en el archivo; y el respaldo ZIP permite conservar una copia de los datos y del índice.

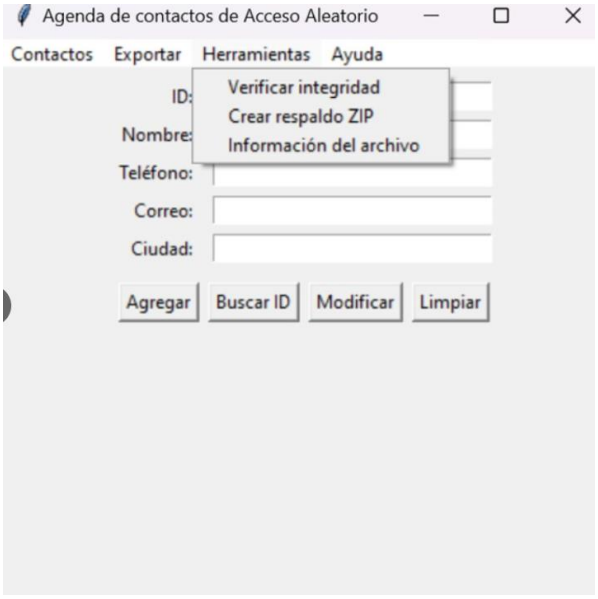
Estas herramientas permiten organizar, consultar, modificar y verificar información con funciones sencillas de Python.

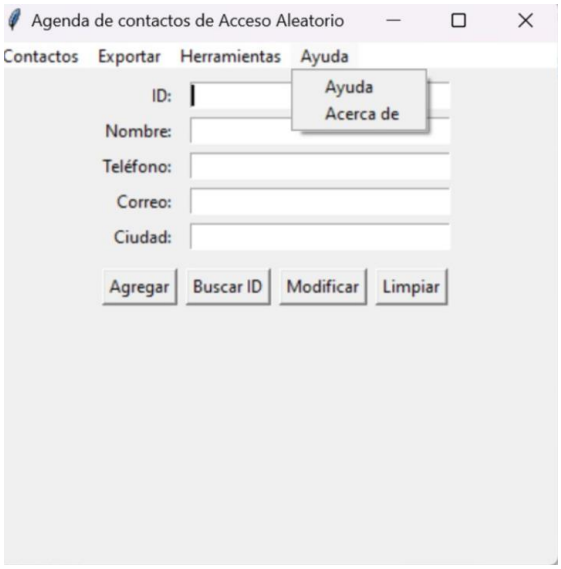
9. Capturas de funcionamiento



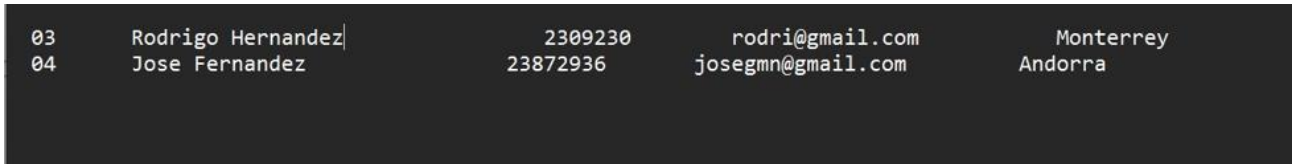


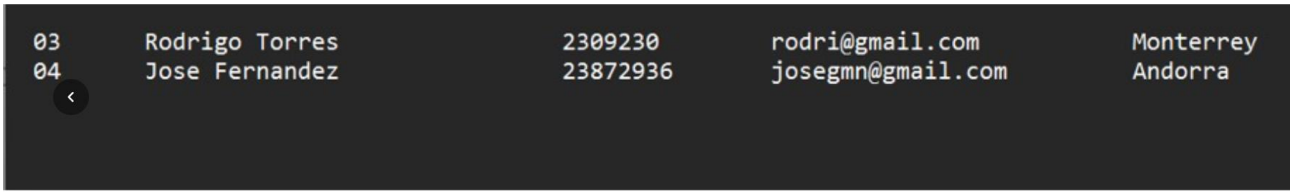





Agenda de contactos de Acceso Aleatorio
Contactos
Exportar
Herramientas
Ayuda
Ayuda
Acerca de
ID:
Nombre:
Teléfono:
Correo:
Ciudad:
Agregar
Buscar ID
Modificar
Limpiar


reportede_contactos.pdf
1 / 1
100%
Reporte de Contactos
ID: 03 | Rodrigo Torres | Tel: 2309230 | rodri@gmail.com | Monterrey
ID: 04 | Jose Fernandez | Tel: 23872936 | josegmn@gmail.com | Andorra


03 Rodrigo Hernandez 2309230 rodri@gmail.com Monterrey
04 Jose Fernandez 23872936 josegmn@gmail.com Andorra


03 Rodrigo Torres 2309230 rodri@gmail.com Monterrey
04 Jose Fernandez 23872936 josegmn@gmail.com Andorra

```
{
  "fecha_generacion": "2026-10-27 19:58:01",
  "total": 2,
  "contactos": [
    {
      "id": "03",
      "nombre": "Rodrigo Torres",
      "telefono": "2309230",
      "correo": "rodri@gmail.com",
      "ciudad": "Monterrey"
    },
    {
      "id": "04",
      "nombre": "Jose Fernandez",
      "telefono": "23872936",
      "correo": "josegmn@gmail.com",
      "ciudad": "Andorra"
    }
  ]
}
```